I like to use a bit of a trick when it comes to overriding the default Identity UI like this. In many cases, you don't want to change the *page handlers* for the Razor Page—only the Razor *view*. You can achieve this by deleting the Register.cshtml.cs `PageModel` file, and pointing your newly scaffolded .cshtml file at the *original* `PageModel`, which is part of the default UI NuGet package.

The other benefit of this approach is that you can delete some of the other files that were autoscaffolded. In total, you can make the following changes:

- Update the `@model` directive in Register.cshtml to point to the default UI `PageModel`:

```
@model
➥ Microsoft.AspNetCore.Identity.UI.V5.Pages.Account.Internal.RegisterModel
```

- Update Areas/Identity/Pages/_ViewImports.cshtml to the following:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

- Delete Areas/Identity/Pages/_ValidationScriptsPartial.cshtml.
- Delete Areas/Identity/Pages/Account/Register.cshtml.cs.
- Delete Areas/Identity/Pages/Account/_ViewImports.cshtml.

After making all these changes, you'll have the best of both worlds: you can update the default UI Razor Pages HTML without taking on the responsibility of maintaining the default UI code-behind.

TIP In the source code for the book, you can see these changes in action, where the Register view has been customized to remove the references to external identity providers.

Unfortunately, it's not always possible to use the default UI `PageModel`. Sometimes you *need* to update the page handlers, such as when you want to